# 山东大学 计算机科学与技术 学院

## 人工智能导论 课程实验报告

| 学　　号　　：<br>202200130077 | 姓名：　薛润泽 | 邮箱： |
|---|---|---|

**实验题目：实验 2 A*算法求解八数码问题**

**实验过程：**

（记录实验过程、遇到的问题和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。）

1.实验过程：求解该八数码问题需要使用 A*算法，主要包括广度优先搜索和估值函数。

（1）BFS 广度优先搜索的实现主要使用 STL 队列（quene）。将每一步的搜索结果压入队列中，每次搜索时将队首的数据弹出使用搜索，依次进行可以满足一层一层的广度优先搜索。

（2）优先队列（priority_queue）：我们需要使用估值函数判断每一个数据的“值”的大小，根据数据的值有顺序的放入队列，所以我们需要使用优先队列。优先队列可以根据我们的运算规则自动将压入队列中的数据排序，方便每次搜索时使用的数据都是估值函数计算的值最高或者最低的数据。

（3）重载运算符：我使用了结构体来存储每次计算返回的状态

```
struct number{
    int hx;
    int cnt;
    int step;
    int nots;
};
```

其中 hx 用来存储估值函数计算的值。Cnt 以一个 9 位数的整数存储当前的 3×3 九宫格。Step 存储最短路径数。Nots 存储运算结点数。

同时我们需要重载运算符

```
bool operator<(const number &a,const number &b){
    return a.hx >= b.hx;
}
```

使用结构体中 hx 来比较数据的优先级，因为使用了九宫格中每个数到目标距离的曼哈顿距离之和作为估值函数，所以期望一个 hx 值更小的数据，队列中以升序排序。

（4）估值函数：计算当前状态中每个点到目标状态的曼哈顿距离之和，函数返回值越小越优先。因为该估值函数 f(x)=g(x)+h(x),其中 f(x)是总值，g(x)是该状态到目标节点 x 的移动代价，h(x)是目标结点 x 到目标状态 aim 的移动代价。因为每次从现在状态移动到目标结点 x 的步数恒为 1，即 f(x)=h(x)+1，所以忽略 g(x)，近似 f(x)=h(x),每次求算所有点到目标状态的曼哈顿距离之和。

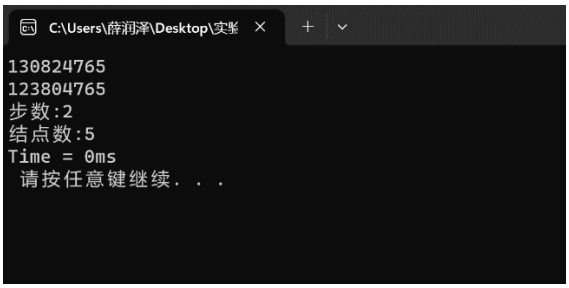
2.遇到的问题

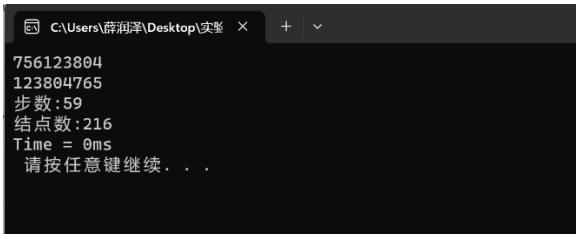
在写 A*算法解决八数码问题之前，并没有学习过广度优先搜索，所以需要了解学习广度优先搜索。为了简化过程，在实验中学习了 STL 中的队列函数用法以及重载运算的写法。

3.实验结果

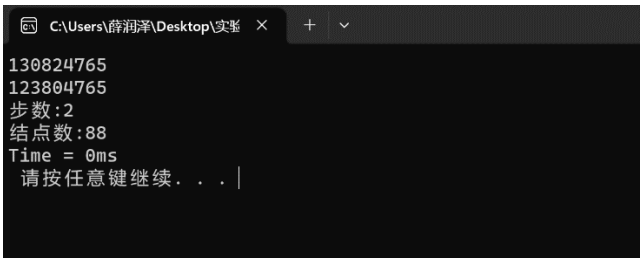
成功写出了八数码问题的 A*算法解决方法。

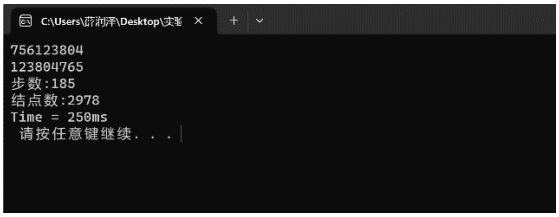
使用了估值函数（A*算法）：




未使用估值函数（广度优先搜索）




以输入 756123804 为例

| | 启发函数 h（x） | |
|---|---|---|
| | h（x）=0 | h(x)=∑[abs(xi-xm)+abs(yi-ym)] |
| 初始状态 | 756123804 | 756123804 |
| 目标状态 | 123804765 | 123804765 |
| 最优解 | 185(steps) | 59(steps) |
| 扩展结点数 | | |
| 生成结点数 | 2978 | 216 |
| 运行时间 | 250ms | 0ms |

结果分析与体会：

结果可以看到，使用了估值函数的启发式 A*算法的结点数比未使用估值函数的盲目广度优先搜索的结点数要少很多，最优步数也会减少，同时运行时间也会降低。当我们有足够的信息可以写出估值函数时，可以尝试使用 A*算法来优化代码。